

MEDS Lab Module 4

Section 12: Integration Task

Task 17: Synchronous FIFO Controller

July 23, 2026

1. EDA Playground Link

The complete design and testbench code for the parameterized Synchronous FIFO Controller can be found at the following shareable link:

- https://edaplayground.com/x/DK_W

2. Schematic Diagram

Below is the top-level block diagram illustrating the interfaces of the FIFO controller.

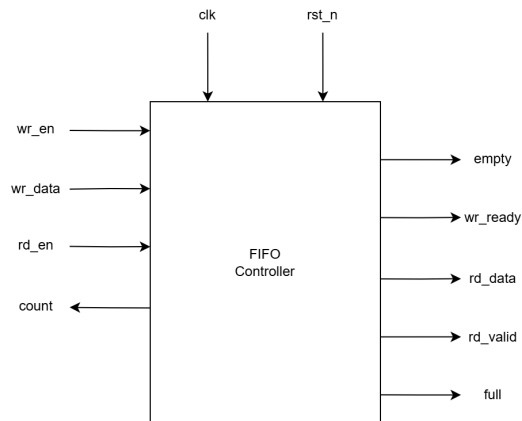


Figure 1: Top-level block diagram of the Synchronous FIFO Controller.

3. Terminal Output Screenshot

The following screenshot displays the simulation log. The self-checking testbench verifies all critical operational modes, including reset, filled, overflow protection, read/write matching, emptied, underflow protection, and simultaneous read/write behavior.

```
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 23 10:12 2026
[PASS] Reset: Empty
[PASS] Filled: Full flag
[PASS] Overflow protected
[PASS] Read matches write
[PASS] Read matches write
[PASS] Read matches write
[PASS] Read matches write
[PASS] Read matches write
[PASS] Read matches write
[PASS] Read matches write
[PASS] Read matches write
[PASS] Emptied: Empty flag
[PASS] Underflow protected
[PASS] Simultaneous R/W Data
[PASS] Simultaneous R/W Count
Done!
```

Figure 2: Simulation log output from EDA Playground verifying all FIFO operations.

4. Waveform Screenshot

The waveform visualizes the internal states, pointer logic, and output flags as the FIFO is filled, emptied, and subjected to simultaneous read/write requests.

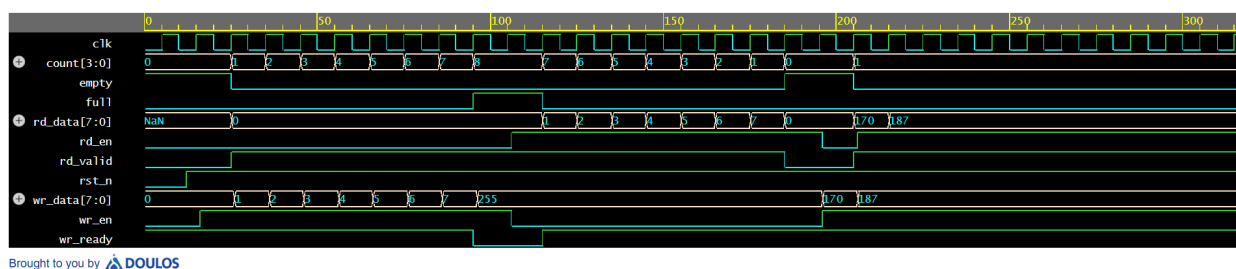


Figure 3: EPWave waveform confirming pointer updates, count tracking, and full/empty flags.

5. Explanation

This task involved designing a fully parameterized synchronous FIFO controller combining sequential and combinational design principles . The module utilizes `ADDR_WIDTH` and `DATA_WIDTH` parameters to dynamically size the internal memory depth and data width.

The design tracks operations using separate `wr_ptr` and `rd_ptr` registers, which function as modulo counters wrapping around naturally due to their fixed bit-width. A dedicated `count` register explicitly tracks the number of elements currently stored in the memory array. This `count` value directly drives the combinational logic for the `empty` (`count == 0`) and `full` (`count == DEPTH`) status flags.

Flow control is strictly enforced: writes are permitted only if `wr_en` is asserted and the FIFO is not full, preventing data corruption via overflow. Reads are permitted only if `rd_en` is asserted and the FIFO is not empty, preventing underflow. The primary state updates occur within a sequential `always_ff` block utilizing an active-low asynchronous reset (`rst_n`) for immediate initialization, satisfying all requirements of the specification.